Design and Implementation of a Compiler with Machine-Learning-Guided Optimization Selection

Kedar Sedai
kedar.sedai05@gmail.com

***Abstract*—The entire design, implementation, and evaluation of MiniLang, a research compiler for a minimum statically typed imperative language, are described in this work. MiniLang was created from base up in C++17 to facilitate machine learning-guided optimization and end-to-end research of compiler development. Handwritten Lexical analysis, recursive-descent parsing, two-pass semantic analysis, lowering to a flat High-level intermediate Representation (HIR), configurable rule-based optimizations, textual LLVM IR emission, and an ML-ready optimization advisor with evaluation harness comprise the system's implementation of a traditional multi-phase pipeline.**

We describe every compilation phase in implementation-level detail, including token design, abstract syntax tree (AST) structure, semantic rules, HIR instruction set, three optimization passes, LLVM code generation strategy, and a feature-driven advisor that selects optimization pipeline using heuristics, JSON plans or external Python interface. Experimental results in benchmark programs show that optimization reduces the HIR instruction count by up to 63% ($22 \rightarrow 8$ instruction) and LLVM IR size by 41% ($34 \rightarrow 20$ lines) on constant-heavy workloads, while preserving correct runtime behavior(verified by process exit codes.) Minilang is intended as a reproducible research and teaching traditional compiler engineering and modern ML-for-compilers research

***Index Terms*—Compilers, intermediate representation, LLVM, static analysis, code optimization, machine learning, pass ordering**

I. Introduction

A. Motivation

Optimizing compilers transform source programs through dozens of analysis and transformation passes. Deciding which passes to run, and in what order has been tuned manually for decades in systems such as GCC and LLVM [1], [2]. Recently, machine learning(ML) has been applied to predict profitable compiler decisions-including pass ordering, heuristics flags, and inlining thresholds-from program features [3], [4]However, experimenting with ML-guided optimization typically requires navigating industrial-strength compiler infrastructures whose complexity obscures the relationship between front-end structure, IR design, and optimization outcomes.
Minilang addresses this by providing a complete but minimal compiler where every phase is inspectable, independently executable, and instrumented for measurement. This project follows a staged research methodology:

- Language design - define a small typed language sufficient for control flow, functions, and arithmetic.
- Front-end construction- implement lexer, parser, and semantic analyzer with explicit error reporting.
- IR design - lower AST to flat HIR with explicit temporaries and structured control flow.
- Rule-based optimization - implement classic local optimizations at HIR level.
- Code generation - emit human-readable LLVM IR for execution via Clang.
- ML advisor integration - extract static features, select pass pipelines, and evaluate against baselines.

B. Research Questions

This works investigates:

- RQ1: Can a minimal imperative language support a full, pedagogically transparent compilation pipeline comparable to textbook architecture [1], [5]?
- RQ2: Is a flat HIR with explicit temporaries a suitable layer for both rule-based optimization and feature extraction?
- RQ3: Can static HIR metrics drive selective application of optimization passes with measurable IR reduction?

C. Contributions

- Formal specification of the MiniLang language (lexical, syntactic, and semantic rules).
- Complete architectural documentation of a C++17 compiler with six standalone driver tools.
- HIR specification with lowering algorithms for expressions, conditionals, and loops.
- Three HIR optimizations with fixed-point iteration and statistics.
- ML-ready advisor framework:14-dimensional feature vector, pass, registery, heuristic/JSON/Python modes, and minilang_eval harness.
- Empirical evaluation demonstrating IR reduction and correctness on benchmark programs.

D. Paper Organization

Section II reviews related work. Section III defines MiniLang. Section IV presents system architecture. Sections V-X detail each compilation phase. Section XI covers the ML advisor. Section XII presents evaluation. Section XIII-XIV discuss limitations and conclude.

II. Related Work

A. Compiler Construction Foundations

The classic compiler pipeline- lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization and code generation- is established in Aho et al. [1] and Cooper and Torczon [2]. MiniLang adopts this structure verbatim, using a hand-written scanner and recursive-descent parser rather than generator tools(Flex/Bison) to maximize transparency for readers and students.
Appel [6] and Muchnick [7] emphasize the role of intermediate representation in decoupling front-end language features from back-end code generation. Our HIR follows the principle: it is lower than AST(flat instruction list) but higher than LLVM (no SSA no target- specific details).

B. LLVM as a Back-end

Lattner and Adve introduced LLVM as a modular compilation framework with SSA-based intermediate representation and aggressive optimization infrastructure [2]. Rather than linking against the LLVM C++ API initially, MiniLang emits textual LLVM IR (.ll files), which Clang consumes for native code generation, This approach mirrors the LLVM Kaleisoscoe tutorial [8] and keeps the research artifact readable in published papers.

C. Machine Learning for Compiler Optimization

Ashouri et al. provide a comprehensive survey of ML applied to compiler autotuning, pass ordering, and heuristic selection [3]. CompilerGym exposes LLVM optimization as a reinforcement-learning environment [9]. Other work learns graph representations of programs for heuristic tuning [4], [5]. MiniLang differs from these systems in scope and interpretability: optimization decisions occur at HIR level with explicit, low-dimensional features (14 integers), making it suitable for classroom demonstration and lightweight ML prototyping without requiring GPU-scale training infrastructure. The advisor is architected as a pluggable policy (heuristic today; ONNX or Python model tomorrow), aligning with survey recommendations for separating feature extraction from inference [3].

D. Educational Compilers

Educational compilers traditionally prioritize clarity over performance. MinLang extends the educational pattern by adding (1) a documented semantic analyzer with symbol tables (2) a serialized HIR dump format, and (3) quantitative evaluation tooling- bridging teaching and research.

III. The MiniLang Programming Language

MiniLang is intentionally small. It supports integer and boolean computation, functions conditionals, and loops. but omits pointers, arrays, help allocation, and I/O builtins.

A. Lexical Structure

Category	Rules
Whitespace	Space, tab, newline; tracked for source locations
Comments	`// to EOL; /* ... */ block comments`
Identifiers	`[A-Za-z_][A-Za-z0-9_]*`
Integers	`[0-9]+` (decimal, signed via unary -)
Keywords	`int, bool, void, if, else, while, return, true, false`
Operators	`+ - * / %`, comparisons, `&& || ! =`
Delimiters	`( ) { } [ ] ; ,`

TABLE I: MiniLang Lexical Rules

The lexer classifies keywords via `classify_keyword()` after scanning identifiers. Multi-character operators (`==`, `!=`, $\leq, \geq$, `&&`, $||$) are recognized using one-character lookahead (match()).

B. Context-Free Grammar

MiniLang grammar is LL(1)-compatible:

```
program     ::= function*

function    ::= type IDENT "(" [ parameters ]
    ")" block

parameters  ::= type IDENT
                ( "," type IDENT )*

block       ::= "{" statement* "}"

statement   ::= block
              | type IDENT "=" expression ";"
              | "if" "(" expression ")"
                  statement
                [ "else" statement ]
              | "while" "(" expression ")"
                  statement
              | "return" [ expression ] ";"
              | expression ";"

expression  ::= logical_or

logical_or  ::= logical_and
                ( "||" logical_and )*

logical_and ::= equality
                ( "&&" equality )*

equality    ::= comparison
                ( ( "==" | "!=" ) comparison )
                    *

comparison  ::= term
                ( ( "<" | "<=" | ">" | ">=" )
                    term )*

term        ::= factor
                ( ( "+" | "-" ) factor )*

factor      ::= unary
                ( ( "*" | "/" | "%" ) unary )*

unary       ::= ( "!" | "-" ) unary
              | call
```

```
call        ::= primary

primary     ::= INT
              | "true"
              | "false"
              | IDENT [ "(" [ arguments ] ")"
                ]
              | "(" expression ")"

arguments   ::= expression
                ( "," expression )*

type        ::= "int" | "bool" | "void"
```

Operator precedence increases toward primary:logical OR (lowest) through multiplicative operators (highest). Unary operators bind tighter than binary operators.

C. Abstract Syntax Tree

The AST uses a tagged-union style with explicit Kind enumerations:

- Expressions: IntLiteral, BoolLiteral, Variable, Binary, Unary, Call.
- Statements: Block, VarDecl, If, While, Return, Expr
- Top level: ($Program \rightarrow vector$) of FunctionDecl

Each node carries a SourceLocation (filename, line, column) for diagnostics. Types are represented by TypeKind: Int, Bool, Void.

Example. The factorial program:

```
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}

int main() {
    int x = factorial(5);
    return x;
}
```

parses into two FunctionDecl nodes with nested IfStmt, ReturnStmt, CallExpr. and VarDeclStmt nodes.

D. Static Semantics

The SemanticAnalyzer performs:
Pass 1 - Function registration: Build global map function_: ($name \rightarrow return\_type, parameter\_types, location$).Reject duplicate definitations.
Pass 2 - Per-function checking: Maintain stack of scope maps (scopes_) for block- enter/block-exit, Rules include:

Construct	Rule
Variable declaration	Initializer required; initializer type must match declared type
Variable reference	Must be declared in enclosing scope (inner shadows outer)
Arithmetic (+, -, *, /, %)	Operands: int; result: int
Comparison	Operands: int; result: bool
Logical(`&&`, `||`, `!`)	Operands: bool, result: bool
if / white condition	Must be bool
Function call	Callee must exist; arify and argument types must match
return	Expression type must match current function return type

TABLE II: Rules:

Error handling: Panic-mode flag suppresses cascading errors after the first error in a function. Messages format as: semantic error at file:line:col:message.

Known limitation: Reassignment (x = expr; without type) is not supported-only type name = expr; declarations. This simplifies semantic analysis for the initial research prototype.

IV. System Architecture

A. Pipeline Overview

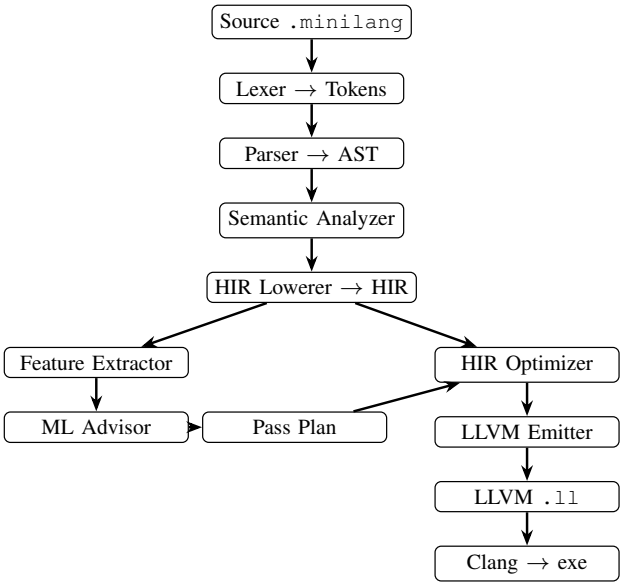

Fig. 1: Data flow.

B. Design Decisions

MiniLang is implemented in C++17 because it aligns with the LLVM ecosystem and is widely used in compiler courses and research. We use a hand-written lexer and recursive-descent parser instead of Flex/Bison so each compilation phase remains easy to read, debug, and explain in a research paper. The following table shows the key design choices.

Decision	Alternatives Considered	Rationale
C++17	Rust, Java	Native LLVM ecosystem; familiar in compiler courses [1]
Hand-written lexer/parser	Flex/Bison, ANTLR	Debuggability; direct mapping to textbook algorithms
HIR before LLVM	Direct AST → LLVM	Isolates front-end optimizations; simpler feature extraction
Textual LLVM IR	LLVM C++ API	Human-readable artifacts; faster research iteration
Separate drivers per phase	Single MiniLang CLI	Independent testing and paper screenshots per stage

TABLE III: Key Design Choices

V. Phase 1: Lexical Analysis

A. Scanner Architecture

The Lexer class maintains:

- source_, filename_ - input buffer and path
- current_, start_, - scan pointers
- line_, column_ - position tracking
- had_error - error flag

Algorithm - Tokenization loop

- skip_whitespace_and_comments()
- If at end - return EOF token
- Record token start location
- If letter or '-' - scan_identifier() - keyword or Identifier
- If digit - scan_number() - IntLiteral with int_value
- Else switch on punctuation with lookahead for multi-char ops
- On unknown char - Invalid token + error message

B. Comment and Whitespace Handling

skip_whitespace_and_comments() loops until non-skippable input:

- Whitespace: advance without emitting tokens
- //: consume until newline
- /*: consume until */ (non-nested)

C. Token Representation

Each Token contains:

- TokenKind (enum: KwInt, Plus, EqEq,...)
- lexeme (source spelling)
- SourceLocation
- optional<int64_t> int_value for integer literals

This design avoids re-parsing lexemes in the parser for numeric values.

D. Error Reporting

Invalid characters produce TokenKind::Invalid with descriptive messages. The lexer driver sets had_error_; compilation aborts with exit code 2 if lexer or parser errors occured.

VI. Phase 2: Syntactic Analysis

A. Parser Strategy

The Parser uses recursive descent: each grammar production maps to a parse_*() method. The parser holds cuurent_ token and provides:

- match(kind) - consume if current token matches.
- consume(kind, message) - require token or error `synchronize()` — Panic-mode recovery at statement boundaries (`;`, }, keywords)

B. Expression Parsing

Expression parsing follows the precedence chain: `parse_expression()` → `parse_logical_or()` → ... → `parse_primary()` Each level handles left-associative binary operators by looping:

```
// Conceptual structure for parse_term()
left = parse_factor();
while (match(Plus) || match(Minus)) {
    op = previous();
    right = parse_factor();
    left = make_binary(op, left, right);
}
return left;
```

Function calls use `finish_call()`: after parsing identifier foo, if (follows, parse argument list.

C. Statement and Function Parsing

- parse_function() - type, name, parameters, block body
- parse_statement() - dispatches on keyword or type lookahead
- parse_var_decl() - requires = and expression initializer
- parse_if_statement(), parse_while_statement() - requires parenthesized condition

D. Parser Output

Successful parse yields Program AST. dump_program() provides indented tree output for debugging and paper listings.

VII. Phase 3: Semantic Analysis

A. Symbol Tables

Two symbol table structure:

- **Function table** (`unordered_map<std::string, FunctionSymbol>`) — global, populated in the registration pass.
- **Variable scopes** (`vector<unordered_map<string, VariableSymbol>>`) - stack; enter_scope() / exit_scope() on blocks and function entry.

Lookup walks scopes innermost-first, supporting shadowing.

B. Type Checking Algorithm

`check_expression()` returns TypeKind recursively.

AST node	Action
IntLiteral	Return Int
BoolLiteral	Return Bool
Variable	Lookup; error if undefined
Binary	Check operands per operator class; return result type
Unary	- requires int; ! requires bool
Call	Resolve function; check each argument type

TABLE IV: Type Checking

`check_statement()` validates control-flow conditions and return types against current_return_type_.

C. Validation Tests

Negative test programs verify error detection:

- semantic_bad_undefined.minilang - use of undeclared variable.
- semantic_bad_type.minilang - type mismatch in expression

These are essential for demonstrating semantic soundness in the evaluation section.

VIII. Phase 4: HIR Lowering

A. Rationale for HIR

The AST is tree-structured and implicit about evaluation order and storage. HIR makes order explicit:

- Every subexpression result is stored in a numbered temporary (%t0, %t1, ...)
- Locals are named and accessed via LoadLocal / StoreLocal.
- Control flow uses explicit labels and branches.

This flat from simplifies data-flow analysis optimization [1].

B. HIR Instruction Set

Opcode	Fields	Semantics
ConstInt	dest, int_value ;	`dest` ← integer constant
ConstBool	dest, bool_value	`dest` ← boolean constant
LoadLocal	dest, local_name	`dest` ← memory[local]
StoreLocal	local_name, lhs	`memory[local]` ← temp(lhs)
Unary	dest, lhs, unary_op	dest ← op(temp(lhs))
Binary	dest, lhs, rhs, binary_op	dest ← temp(lhs) op temp(rhs)
Call	dest, callee, args[]	dest ← callee(args)
Ret	lhs	return temp(lhs) or void
BrCond	lhs, then_label, else_label	branch on temp(lhs)
Jump	jump_label	unconditional goto
Label	jump_label	branch target

TABLE V: HIR Instruction Set

Each HirFunction store: name, return type, parameters, locals, list, local_types map, and ordered instruction vector.

C. Lowering Algorithms

Expressions: `lower_expression()` allocates `dest = fresh_temp()`, emits one defining instruction, returns dest temp id.

Variable declarations: Evaluate initializer → StoreLocal.

If-else: Classic structured lowering:

- Evaluate condition → cond_temp
- BrCond, cond_temp, then_label, else_label
- Emit then_label: lower then-branch; Jump endif
- Emit else_label (if present): lower else-branch
- Emit endif_label

While loops:

- Jump while_cond
- while_cond: evaluate condition; BrCond, body, end
- while_body: lower body; Jump while_cond
- while_end:

This produces back-edges detectable by the feature extractor as loop_hint_count.

D. Example HIR Fragement

For int x = 2 + 3; inside a function, lowering produces approximately:

```
%t0 = const_int 2
%t1 = const_int 3
%t2 = add %t0, %t3
store_local x <- %t2
```

After constant folding, %t2 may collapse to const_int 5.

IX. Phase 5: HIR Optimization

A. Pass Pipeline Architecture

HirOptimizer accepts a PassPlan:

```
struct PassPlan {
   vector<OptPassKind> passes;  // execution
       order
   int max_iterations = 3;      // fixed-point
        bound
};
```

Registered Passes:

- ConstantFold
- AlgebraicSimplify
- DeadTempRemove

The optimizer iterates up to max_iterations over all functions until no pass reports changes-a standard fixed-point scheme [2].

B. Constant Folding

For each Binary or Unary instruction, the optimizer locates defining instructions for operates via `find_def(temp, func, before_index)`. If operands are ConstInt or ConstBool, the operation is evaluated at compile time and replaced with a constant instruction.

Safety: Division and modulo by zero are not folded (avoid compile-time crash).

Statistics: Each successful fold increments `stats.constant_folds`.

C. Algebraic Simplification

Currently implements identify `x * → 0` (and semantic 0 * x) regardless of whether x is constant. This pairs with constant folding on expressions like `1 + 1 * 0`.

D. Dead Temporary Removal

Algorithm:

- Build used set: scan all instructions for reference in lhs, rhs, args,
- Filter instructions: drop pure computations whose dest is not in used
- Preserve control-flow of dead code elimination of temporaries [1].

E. Optimization Statistics

OptimizationStats tracks folds, algebraic simplifications, and dead temps removed-reported by minilang_hir and minilang_compile drivers for reproducible experiments.

X. Phase 6: LLVM IR Code Generation

A. Emitter Design

LlvmEmitter walks optimized HIR and prints textual LLVM IR to an ostream. Key mappings:

HIR concept	LLVM emission
int / bool	i32 / i1
Local variable	%name.addr = alloca i32 in entry block
Temporary %tN	SSA value %tN
add, mul, ..	add i32 %t0, %t1
Comparisons	icmp, eq, etc.
BrCond	br i1 %cond, label %then, label %else
Call	call i32 @name(...)

TABLE VI: Emitter Design

Header emission includes module comment and target triple = ”unknown-unknown-unknown”.

B. Entry Block and Alloca

minilang_compile supports:

```
minilang_compile program.minilang -o out.ll --
    run
```

which invokes:

```
clang out.ll -o build/a.out -Wno-override-
    module
./build/a.out
```

Exit code is decoded with `WEXITSTATUS()` on POSIX systems-programs communicate results via return from main.

XI. ML-Guided Optimization Advisor

A. Motivation

Running all passes on every program wastes compile time when no opportunities exist(e.g., tiny programs). Ml-guided compilation research suggests predicting pass relevance from program features [3], [9]. MiniLang implements an advisor that outputs a PassPlan before optimization.’

B. Feature Extraction

`extraxt_hir_features(HirModule` computes 14 static metrics:

Feature	Meaning
function_count	Module size
instruction_count	Total HIR instructions
binary_op_count	Arithmetic/logical ops
unary_op_count	Negation/logical-not
call_count	Function calls
branch_count	Conditional branches
jump_count	Unconditional jumps
label_count	Basic block labels
const_int_count	Integer literals
const_bool_count	Boolean literals
load_store_count	Memory traffic
local_count	Local variables
max_temps_per_function	Peak temporaries
loop_hint_count	Back-edge heuristic

TABLE VII: HIR feature vector

`hir_features_to_vector()` converts to vector<double> for ML models. –dump-features prints human-readable values.

C. Advisor Modes

Mode	Mechanism
Heuristic	Rule-based mapping(default)
JsonPlan	Load ”passes”:[...], ”max_iterations”: N from file
PythonScript	popen(”python3 script.py ’<json_features>’”) → parse JSON plan

TABLE VIII: Advisor Modes

The python path enables plugging pre-trained models(scikit-learn, ONNX export, etc.) without recompiling the compiler [3]

D. Heuristic Policy

Rules(implemented in MlAdvisor::heuristic_recommend):

- If `instruction_count <= 8` and binary_op_count → dead_temp_remove only
- If `binary_op_count >= 4` OR const_int_count ≥ 3 → add constant_fold
- If `binary_op_count >= 2` → add algebraic_simplify
- If `max_temps_per_function >= 3` OR loops/branches → add dead_temp_remove
- If no pass selected → full pipeline (all three passes)

Each decision sets last_explanation() for logging and paper traceability.

E. CLI Integration

```
--opt none      # PassPlan empty     no
    optimization
--opt all       # All passes, 3 iterations
--opt advised   # Advisor selects passes
--advisor-plan tests/advisor/sample_plan.json
--advisor-python scripts/advisor_heuristic.py
```

F. Evaluation Harness

`minilang_eval` compiles each input under none, all, and advised, reporting HIR sizes, LLVM line counts, optimization stats, and selected plan-supporting RQ3 measurement.

XII. Experimental Evaluation

A. Experimental Setup

- Platform: macOS/Linux, C++17, Clang backend
- Build: make compile eval
- Benchmarks: factorial.minilang, helloWorld.minlang
- Metrics: HIR instruction count (pre/post opt), LLVM IR line count, per-pass stats
- Correctness: –run exit codes compared to manual calculation.

Program	Strategy	HIR I→O	LLVM lines	Folds	Alg	Dead	Pass Plan
factorial	none	21→21	37	0	0	0	-
factorial	all	21→21	37	0	0	0	full
factorial	advised	21→21	37	0	0	0	full
opt_demo	none	22→22	34	0	0	0	-
opt_demo	all	22→8	20	7	0	14	full
opt_demo	advised	22→8	20	7	0	14	full

TABLE IX: Experimental Evaluation

XIII. Analysis

Constant-heavy code (opt_demo): Source contains (2+3)*(10-4)+(8/2) and (1+1)*0. Constant folding collapses subexpressions; dead temp removal eliminates 14 unused temporaries. HIR reduced 63% LLVM IR reduced 41%.

Recursive code (factorial): Values depend on runtime parameter n; no compile-time folding possible. Advisor correctly still selects full pipeline but statistics remain zero-demonstrating soundness (no incorrect transforms).

Correctness:

- factorial(5) → exit code 120
- opt_demo → (5*6+4)+0 = 34 → exit code 34

A. Discussion of RQs

- RQ1: Yes - full pipeline implemented with six documented phases.
- RQ2: Yes - HIR supports opts and 14-feature extraction with clear semantics.
- RQ3: Yes - measurable IR reduction on suitable benchmarks; advisor framework operational.

B. Threats to Validity

- Small benchmark set; not representative of production workloads.
- Advisor uses heuristics, not a trained neural network-ML is architectural readiness, not novel model contribution.
- LLVM opt pipeline not invoked; measured IR is MiniLang emitter output only.

C. Limitations

Current Limitations:

- No assignment statements, arrays, pointers, strings, or print builtin
- No SSA construction at HIR level
- No register allocation or calling-convention beyond simple LLVM @name
- Error recovery limited to panic mode
- Advisor not trained on large corpora

XIV. Conclusion

We presented MinLang, a research compiler documenting every step from lexical analysis through LLVM code generation, augmented with an ML-ready optimization advisor and quantitative evaluation harness. The project demonstrates that a minimal language can support meaningful compiler engineering and optimization research when IR design, pass modularity, and feature extraction are treated as first-class design goals. On constant-heavy benchmarks, optimization achieves substantial IR reduction while preserving semantics; on runtime-dependent code, the optimizer correctly refrains from invalid folds. MiniLang provides a reproducible foundation for teaching compiler construction and experimenting with machine-learning-guided pass selection.

References

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA: Addison-Wesley, 2007.

[2] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, 2004, pp. 75–88.

[3] M. Ashouri, A. Marques, C. Cummins, R. C. Joao, B. Wu, P. Petersen *et al.*, "A survey on compiler autotuning using machine learning," *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–39, 2023.

[4] H. M. G. Welle, B. Wu, C. Cummins, P. Tint, H. Leather, and M. F. P. O'Boyle, "A graph representation for machine learning assisted compiler heuristic tuning," in *Proceedings of the International Conference on Languages and Compilers for Parallel Computing (LCPC)*, 2019, pp. 153–169.

[5] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 2nd ed. Waltham, MA: Morgan Kaufmann, 2011.

[6] A. W. Appel, *Modern Compiler Implementation in C*. Cambridge, U.K.: Cambridge University Press, 2004.

[7] S. S. Muchnick, *Advanced Compiler Design and Implementation*. San Francisco, CA: Morgan Kaufmann, 1997.

[8] C. Cummins, B. Wu, Z. Wang, J. Leather, M. Webster, and H. Leather, "Programl: Graph-based deep learning for program optimization and analysis," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[9] C. Cummins, Z. Wang, J. Leather, M. Webster, S. Sykosch, H. Leather *et al.*, "Compilergym: Robust, performant compiler optimization environments for ai research," in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, 2022.

[10] LLVM Project, "Kaleidoscope: Implementing a language with llvm," https://llvm.org/docs/tutorial/, 2024, accessed: 2026-06-26.